

Analyzing True and False Positives Produced by JSpecify and NullAway

Eric Weng
qgt7zm@virginia.edu
May 8, 2026

Abstract—Null pointer exceptions are among the most common and damaging coding errors. Older languages in Java have no native features to encode nullability. The JSpecify annotation library and NullAway static checker are promising solutions that detect nullability errors in Java programs. In this project, I assess the strengths and weaknesses of these tools compared to standard IDE features on unannotated code. By running these tools, I record the number and reason of nullability warnings. I then conduct manual dataflow analysis to divide all warnings into true or false positives. My work reveals that thorough annotation of the codebase prevents almost all false positives and uncovers a few true positives that unit tests do not cover. Even discounting the new warnings raised, I claim that annotations improve code readability and makes developers more attentive toward nullability.

I. INTRODUCTION

A. Background

Null pointer errors are one of the most frequent and costliest mistakes in programming. The creator of the concept, Tony Hoare, cites his invention of the null pointer as his “billion-dollar mistake,” owing to how a single null dereference can instantly crash a program or trigger undefined behavior [1]. In 2025, MITRE Corporation ranked null pointer errors as the 13th most common Common Weakness Enumeration (CWE) [2]. Since Java is a garbage-collected language, null pointer errors only become even more abundant once other low-level memory errors are removed from consideration.

While unit testing is a successful and popular technique for code verification, static analysis is a more thorough approach that catches faults before they even occur. Newer JVM languages such as Kotlin have built-in operators and types that enforce null safety at compile-time. However, Java still does not have any native syntax for null safety, besides the `Optional<T>` type, which itself may be null. While rewriting entire programs in Kotlin is possible, doing so is costlier and messier than making existing Java code safer. In the past, there have been many such competing solutions for Java, each with different levels of rigor, compatibility, and maintenance. On the other hand, having no community standard risks fragmentation and hampers readability between different projects [3].

Thankfully, an emerging standard for static Java null safety is JSpecify, an annotation library co-developed by leading organizations that produce Java applications [4]. JSpecify allows programmers to mark variables as `@Nullable` (may be null) or `@NonNull` (is never null). Entire packages may

even be marked as `@Nullable`, meaning all variables are `@NotNull` unless explicitly annotated `@Nullable`. Afterward, static checkers such as IntelliJ IDEA and NullAway process JSpecify annotations to perform nullability dataflow analysis. Unlike other checkers, NullAway integrates into the Java compiler and outputs the location and cause of potentially null-unsafe code. At first glance, adopting static checking into all Java projects seems like an obvious solution to null-pointer errors.

B. Significance

Three substantial challenges towards greater adoption of such tools are opportunity cost, false positives, and stability. First, merely applying JSpecify annotations to an existing codebase requires developers to potentially sift through someone else’s years-old code and consider all possible paths and usages of a function. Running NullAway for the first time in a project may uncover a plethora of “new” raised due to improper annotations or checker configuration. Although NullAway claims to display as few false positives as possible [5], every warning still requires debuggers to divert precious time away from building new features or fixing already confirmed bugs. In a sense, software developers must first execute the exact data analysis that these tools are meant to automate.

Furthermore, if NullAway is configured for continuous integration, then nullability warnings may break new builds. As one program maintainer explains, dismissing extraneous warnings (or even true one temporarily) so the code can compile [6] requires manual suppressions or poorly-constructed workarounds that increase technical debt. Too many false positives will cause debuggers to place less trust in future warnings, use the tool incorrectly, or revert to already proven methods such as unit testing. As a result of humans’ resistance to change, several projects miss out on some or all of the benefits of static checking.

Lastly, NullAway and JSpecify require at minimum JDK 17, and JDK 21 to fully utilize all their features. A quick glance in GitHub will reveal a plethora of major projects still relying on runtimes as old as Java 8. Updating the language version requires careful management of external libraries and consideration of deprecated APIs. Carelessly upgrading these environments to a new Java language version could break dependencies and precipitate untold hours of refactoring.

In 2019, the developers of NullAway at Uber conducted a study into NullAway’s overhead and soundness [5]. Using several open-source libraries as benchmarks, the authors demonstrate that NullAway compiles faster than existing static checkers and imposes a reasonable annotation burden on humans. They also show that NullAway produces no false negatives (missed null pointer errors) by categorizing errors in deployment, though no analysis is made for the type of false positives raised during compilation. NullAway accomplishes this zero false negative rate using optimistic assumptions (everything is `@NonNull` unless marked otherwise) in both internal and external code.

However, I believe that the study’s methodology does not match developers’ demonstrated preferences. Incremental compilation and faster hardware can mitigate any decreases in compile speed. Moreover, false negatives could potentially be caught by existing test suites, which developers would supplement—not replace—with static checking. In contrast, false positives do not reveal actionable faults in the code and waste time and effort. Additionally, the writers only test NullAway on a narrow set of domains (mostly mobile and web), which may not reflect the average Java developer’s experience.

Besides the NullAway paper, I found no research done on JSpecify, despite major companies, including Google, JetBrains, Uber, supporting its development. Since the NullAway paper’s publishing, the JSpecify team has pushed their first stable release in 2024 and NullAway has introduced support for JSpecify annotations and new language features like generics. If anything, these static checker tools have only improved overtime, and past criticisms and experiments may no longer apply.

D. Problem Statement

In this project, I intend to reevaluate whether NullAway and JSpecify raise too many false positives to be beneficial to developers. Namely, I will compare the number of true and false positives raised, count the reasons behind each warning, and determine if project domain affects the distribution of warnings. Similarly to the NullAway study, I will build of benchmark of several open-source projects, pulling from a wider range of domains when possible.

Compared to the study, I foresee my results will be more grounded and immediately useful for average developers unfamiliar with static checker tools. Instead of overselling these tools as silver bullets, I will set realistic expectations for programmers to better help them maintain trust with new tools and better understand their use cases. My method will also provide a more granular breakdown into true and false positives and how their distributions are affected by domain. Users will learn what warnings they should and should not expect to uncover, and where they should be more careful when annotating. To my knowledge, this work will also be first empirical and systematic study of JSpecify.

A. Overview

For my project, I conduct static analysis on Java projects that use JSpecify annotations. I approach my experiments from the perspective of a Java developer who has read the NullAway GitHub page [7] and is planing to follow the recommended procedures. First, my baseline configuration simulates an existing project that has not yet integrated any annotations. As such, I remove all annotations and only use IntelliJ IDEA’s out-of-the-box Code Inspection tool to find nullability issues. Second, my experimental treatment represents the same project that has adopted JSpecify and NullAway. To this end, I restore all annotations and enable NullAway compiler warnings on JSpecify annotations.

For each project and configuration, I count the number of warnings raised and determine whether they are true or false positives. I keep my analysis conservative, classifying uncertain warnings as true positives. Then, I investigate the reasons behind each warnings by manually tracing usages for possible null pointer errors. NullAway and IntelliJ provide the locations of warnings and usages, allowing me to easily locate function calls and view their signatures. To ensure a fair analysis of true negatives, I also remove any instances of manual suppressions relating to nullability issues.

B. Methods

To start, I create my baseline configuration, I commenting out JSpecify annotations (`@Nullable`, `@NotNull`, and `@NonNullMarked`) and manual suppressions (`@SuppressWarnings(values)` with parameter containing substring “null”). Commenting instead of deleting allows me to preserve line numbers and see where annotations used to be. For annotations in the middle of a line, I use block comments (`/*...*/`), and elsewhere I use one-line comments (`//...`). I use version control to easily revert my changes later on. To analyze a pre-JSpecify project, I run IntelliJ IDEA’s Code Inspection tool on the package(s) of the project I am evaluating. I only select inspections containing the keyword “null” and that are described as potentially leading to an error and disable warnings about code style or redundant checks. At this point, I am not using NullAway because it does not reliably provide warnings when annotations are absent.

For my experimental configuration, I restore all removed annotations and (re)enable NullAway by modifying the build system configuration file. Interestingly, most of my benchmark projects were already using NullAway or its required dependency, Error Prone. However, the nullability checks may have been disabled due to the maintainers not wanting to deal with all the error messages. I add NullAway and Error Prone as compiler plugins inside the root Maven `pom.xml` file and export the necessary JVM packages. I then configure Error Prone to only carry out NullAway checks and ignore any test code. I also configure NullAway to use JSpecify mode and only analyze `@NonNullMarked` packages. Lastly, I set the severity level to `ERROR`, which fails the compilation

if unsafe code is detected, unlike `WARN`, but would also break continuous integration on false positives. I reason that error messages are less likely to be ignored. Once configured, NullAway can be run by compiling the (sub)project from scratch (i.e. deleting previous builds) and viewing the log. I also run IntelliJ’s Code Inspection since there is no reason for a developer to not use every tool at their disposal.

Following, I perform my analysis of the static checker results. For each distinct warning or error raised, I record the filename, method/variable name, line number, source (IntelliJ/NullAway), and reason cited by the checker. Next, I classify each warning as a false positive, true positive, or uncertain. A *false positive* means that I found no usages that would result in a null pointer exception, and the warning can trivially be resolved with better annotation. A *true positive* means there is a clear path that would result in a null dereference. *Uncertain* means the programmer’s intention was unclear to me or I could not definitively rule out errors due to lack of big-picture understanding of the code. I export my final results to a CSV file to facilitate data processing. Below is an example of how I would de-annotate code and what warnings might be raised:

```
// Baseline
/*@Nullable*/ Course findCourse(
    /*@NonNull*/ String courseCode);

// Experimental
@Nullable Course findCourse(
    @NonNull String courseCode);

// Unsafe Usages
findCourse(null); // (1)
findCourse("CS 101").getName(); // (2)
```

The method `findCourse` accepts a non-null string and returns a nullable `Course` object. A checker should report Usage 1 as unsafe, since passing null to a non-null parameter might result in a null pointer exception (if the method checks the string’s length). Likewise, a checker should also report Usage 2 as unsafe since dereferencing a nullable object without a prior check could crash the program.

C. Limitations

The first major limitation of my approach is that adding annotations inevitably alters code style and structure. Although I could compare releases before and after annotations are applied, I want to minimize the semantic differences and likelihood that other issues were fixed. I also only choose projects already annotated since I lack the domain knowledge or familiarity to correctly apply them myself. Some projects in the process of migrating to JSpecify could potentially use other annotation libraries simultaneously. My research does not test if their presence changes anything; however, this is something for project maintainers to consider. Since my solution is meant to uncover warnings raised after initial migration, I do not run much dynamic analysis typically done by programmers afterward.

Moreover, the analysis is easily the most error prone (no pun intended) step of my project. Naturally, static analysis is difficult; if I discovered a quick and foolproof way to perform this step, then I would have probably won an award for solving some form of the halting problem. Methods or variables could have hundreds of usages for me to trace, meaning I could have overlooked an unsafe usage. Additionally, my opinion of each warning reason also changed over time (even since my project proposal) as I analyzed more projects. I am also assuming that the developers’ intentions are correct—that their suppressions and annotations match the semantics of the program and are not placed to silence warnings. I do my best to balance the quantity of warnings recorded and quality of my analysis; many real-world developers face the same pressure and lack of resources.

Therefore, to prove that my classification strategy is sound, I select one interesting false and true positive from each project as “case studies,” focusing on deeper faults when possible. For each case study, I write a unit test that executes one faulty path or demonstrate why no paths are faulty in addition to my usual dataflow analysis. If there are no useful tests to write, then I rewrite the problematic code with the necessary annotations. I include the discussion of each case study example within my evaluation section; the relevant source code and tests are included my deliverable for further study (<https://github.com/qgt7zm/software-analysis-project>).

III. EVALUATION

A. Benchmarks

TABLE I
BENCHMARK LIBRARIES

Library/Packages	Category	LoC	Annot.	Warn.
compile-testing (all)	Compilation	5,092	94	21
guava (.base)	Utilities	13,806	363	35
jsoup (.parser)	HTML	9,221	54	96
log4j-core (.core, .core.impl)	Logging	7,826	52	122
Average	N/A	8,986	141	69

For my benchmarks (Table I), I selected four open-source Java libraries from the Maven Repository of different domains. I counted an average of about 9,000 lines of code (LoC), 140 annotations, and 70 warnings. Larger projects did not correlate with more annotations or warnings, since annotation standards vary between developers. Since these projects were very large, I limited my analysis to the most important sub-modules and packages. Unfortunately, the benchmarks from the NullAway experiments did not use JSpecify, so I could not reuse them. To ensure my results were deterministic, I chose stable release milestones, viewable as GitHub tags. Similarly, I used the same version of Java and static checker tools and libraries across all projects.

B. Number of Warnings per Configuration

Research Question 1: How many true and false positives do NullAway and JSpecify introduce?

TABLE II
WARNINGS RAISED BY CHECKERS

Warning	Baseline	Experimental		
		Total	IntelliJ	NullAway
False positive	114	120	87	80
True positive	17	23	17	11
Total	131	143	104	91
% False positive	87.0%	83.9%	83.7%	87.9%

Table II lists the warnings found in total or by each tool for the baseline (unannotated) and experimental (annotated) groups. Individually, IntelliJ and NullAway raise fewer warnings of all types on annotated code, but combined they raise more of each. Annotating code does lower the proportion of false positives by 3.1%, but with a hundred or so warnings, this decrease is barely a handful. For each tool in isolation, annotations eliminate uncertainty over whether values can be null, causing a net decrease in false positives. Nonetheless, annotations raise new false positives via incorrect application or catching real errors. I noticed that IntelliJ focuses more on nullable assignments and returns, while NullAway focuses more on pointer dereferences. Both tools show a large overlap in warnings, although the overlap is not the majority. At the moment, one would conclude that introducing JSpecify and NullAway does not effect the number of warnings, but later on, I will argue that these tools drastically reduce false positives once annotations are applied thoroughly and correctly.

C. Reasons for Nullability Warnings

Research Question 2: What are the most common reasons behind nullability warnings?

TABLE III
WARNINGS BY CATEGORY

Warning	Category	Count
False positive	Assertions	20
	Fields	30
	Method signatures	55
	Null assignments	72
	Null dereferences	9
	Null returns	40
	Optionals	8
True positive	Fields	2
	Null assignments	6
	Null dereferences	12
	Null returns	21

I found most warnings had similar reasons, either due to the shared tendencies of programmers or locality within the same file. Analyzing one warning usually meant finding the explanation to several others nearby. As a reminder, NullAway considers every non-annotated value in `@Nullable` as `@NonNull`. Unannotated external code (including the Java API) are also treated nullas -marked, which often lead to

false warnings. In Table III, I classified all warnings into the following broad categories:

- 1) **Assertions:** Null checks before dereferences are a safe way to eliminate null pointer errors. By default, NullAway ignored `assert val != null` statements since not all Java runtimes support the `assert` keyword. It also incorrectly assumed calling `getProperty()` after checking `getProperty() != null` could result in a different result.
- 2) **Fields:** Many programs use instance variables to store state, where null values represent an empty or uninitialized state. False positives arose when fields are late-initialized after the constructor or are cleared and “lazily” re-initialized upon reuse.
- 3) **Method Signatures:** Plenty of method overrides were missing `@Nullable` annotations, suggesting developers only annotated the superclass method. Checkers raised a warning if they saw null being passed to a parameter that was `@NonNull` in the superclass.
- 4) **Null Assignments:** Often, developers simply neglect to annotate nullable fields, perhaps thinking the code is clear enough, leading to a multitude of trivial false positives when those fields are assigned a null value.
- 5) **Null Dereferences:** Dereferencing a nullable value passed through a parameter or returned from a method was a frequent source of both true and false positives. Some warnings were caused by external code automatically being treated as null-marked. Actual exceptions occurred with unexpected call sequences and developers losing track of what was null over large files. For example, a method might assume a resource was still open, meaning no guards would be placed to prevent reading a closed resource.
- 6) **Null Returns:** Some function return types were not annotated correctly, resulting in false positives when the output was assumed to be nullable and no checks were present, or vice versa. In a few cases, subclass methods would return unsafe values when the original author did may not have intended such behavior. Rarely, functions would throw exceptions instead of returning null, which NullAway did not recognize. or override behavior unintended by the original author.
- 7) **Optionals:** The `java.util.Optional<T>` wraps a nullable value and provides the `isEmpty()` method to check if the wrapped value is null. As such, comparing an `Optional<T>` to null is normally incorrect. However, Guava has conversion methods between its own optional type and Java’s, where null checks are fully intended.

Overall, the top two false positive categories across all projects were null assignments and method signatures. Most false positives occurred due to incomplete annotation and interactions with external code. Overriding methods may also raise warnings due to a new author misinterpreting the vague documentation of the parent code. Likewise, the two most

frequent true positive categories were null returns and null dereferences. True positives appeared when different usages are separated by many lines of code and programmers assume others will use their function correctly.

D. Distribution of Warnings by Domain

Research Question 3: How does the domain of a project affect which warnings are raised?

TABLE IV
TOP WARNING CATEGORIES PER LIBRARY

Project	Top False Positive	Top True Positive
compile-testing	Method signatures	Null dereferences
guava	Null returns	Null dereferences
jsoup	Null assignments	Null assignments
log4j-core	Method signatures	Null returns

In general, I did not observe any strong association between a project and the distribution of warnings raised. Table IV shows all the top false and true positive warnings for each positives are ranked among the top three overall. Previously, I thought that different software patterns and architecture might affect the spread of warning categories. Now, I believe that because projects utilize variables, methods, and inheritance, the categories (and reasons) should be mostly homogeneous.

IV. CONCLUSION

At first glance, JSpecify and NullAway did not appear to find more true positives over false positives versus a tool like IntelliJ IDEA. Both checkers detect surface-level bugs accurately but struggle to grasp the high-level semantics of the code. However, more exhaustive annotation would discard the shallow and trivial warnings that composed the majority of false positives. When given context about which variables may be null, tools begin filtering out actual errors from noise.

Second, I discovered most warnings, whether true or false, occur in the same contexts, namely incomplete annotations of variables and methods. I surmise most false positives emerge because developers tasked with using annotation libraries are unfamiliar with how to properly apply them. Even natural-language Javadoc comments leave ambiguity in the intention of a function. Having low or inconsistent standards in an organization further impedes communication. Here lies JSpecify’s greatest strength: annotations allow the code to speak for itself. Annotations move the nullability specification from documentation—which must be maintained alongside the code—into code itself—which can be read immediately. Consequently, humans (and machines) no longer have to bear the burden of remembering what is null and what is not.

Finally, JSpecify and NullAway reveal similar warnings across domains due to shared programming habits. The libraries I selected are maintained by experienced developers, who have sufficiently considered how to handle null safety, and are thus already quite safe. Common trouble spots include imprecise natural-language documentation, usages of unannotated external (or even internal) code, and the inherent complexity of large software projects. While some challenges are

unavoidable, I believe that JSpecify improves communication by allowing developers to treat nullability as part of the code, rather than an afterthought.

Questions that remain open are whether my research is generalizable to the larger Java community and how my analysis can be further improved. My experiments are promising and highlight important trends as demonstrated by some of the most popular Java projects. On the contrary, I also only evaluated one library per domain and skipped over numerous other domains. My small sample size (four libraries) leaves me unable to conduct rigorous statistical tests. Furthermore, critics may contend that I should have gone deeper in my analysis.

Good areas for future research include implementing newer, advanced JSpecify annotations that encode more code semantics and evaluating competing annotation libraries and static checkers. I would also be interested in testing whether Kotlin’s nullable types reduce false positives compared to annotated Java. Given a longer time frame, I would have aimed to produce a separate case study write-up for each warning I found, similar to how actual security researchers or software engineers might create bug reports. I could submit these reports to the project maintainers as well as reannotate the problematic code sections, providing the idea method of validating my analysis. However, such a feat would have been out of scope of this project, as I do not know how long issues and pull requests take to review.

Ultimately, I would recommend using JSpecify and NullAway to a developer. I claim that the optimal scenario to use them is with a new project. Since no test suite is available and no code needs to be retroactively edited, annotating code while writing it will catch faults early. Even on an existing project, I argue that JSpecify and NullAway are worth the tedium of configuring and debugging simply due how they change developer mindsets. These tools prompt authors to think carefully about their intentions express them unambiguously. Once the hurdle of fulling migrating a project is surpassed, these tools will continue to catch defects into the future, even if no more tests are written. Additionally, adopting a standardized compiler plugin and library improves the portability of the static analysis. People who are do not use IntelliJ IDEA will cannot access to the code inspection feature. Also, the developers of these tools are working on new annotations that better account for certain patterns and architectures. I hope that as more developers realize the benefits of adopting these tools, the community can create safer code together.

REFERENCES

- [1] T. Hoare, “Null References: The Billion Dollar Mistake,” Aug. 2009. <https://www.infoq.com/presentations/Null-References-The-Billion-Dollar-Mistake-Tony-Hoare/>.
- [2] MITRE, “2025 CWE Top 10 KEV Weaknesses,” Dec. 2025. https://cwe.mitre.org/top25/archive/2025/2025_key_list.html.
- [3] StackOverflow, “Which @nonnull java annotation should i use? [closed],” June 2021. <https://stackoverflow.com/questions/4963300/which-notnull-java-annotation-should-i-use>.
- [4] JSpecify, “Start Here,” 2026. <https://jspecify.dev/docs/start-here/>.
- [5] S. Banerjee, L. Clapp, and M. Sridharan, “Nullaway: practical type-based null safety for java,” in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2019, (New York, NY, USA), p. 740–750, Association for Computing Machinery, Aug. 2019.
- [6] GitHub, “Proposal: Incorporate Error-Prone,” 2018. <https://github.com/ReactiveX/RxJava/issues/6318>.
- [7] GitHub, “Nullaway: A tool to help eliminate nullpointerexceptions (npes) in your java code with low build-time overhead,” 2018. <https://github.com/uber/NullAway/>.